Лабораторная работа № 4

Линейная алгебра

Пакавира Арсениу Висенте Луиш

Содержание

1 Цель работы **4**

2 Задание **5**

3 Теоретическое введение **6**

4 Выполнение лабораторной работы **7**

4.1 Заданиядля самостоятельного выполнения 14

5 Выводы **24**

Список литературы **25**

Список иллюстраций

4.1 Поэлементные операции над многомерными массивами 7
4.2 Поэлементные операции над многомерными массивами 8
4.3 Транспонирование,след,ранг,определительи инверсия матрицы . 9
4.4 Матричное умножение,единичная матрица,скалярное произведение 10
4.5 Факторизация.Специальные матричные структуры 11
4.6 Факторизация.Специальные матричные структуры 12
4.7 Факторизация.Специальные матричные структуры 13
4.8 Общаялинейная алгебра 14
4.9 Произведение векторов 15
4.10 Произведение векторов 15
4.11 Системы линейных уравнений 16
4.12 Систем линейных уравнений 17
4.13 Систем линейных уравнений 18
4.14 Операции с матрицами 19
4.15 Операции с матрицами 20
4.16 Операции с матрицами 21
4.17 Линейные модели экономики 22
4.18 Линейные модели экономики 23

1 Цель работы

Основной целью работы является изучение возможностей специализированных пакетов Julia для выполнения и оценки эффективности операций над объектами линейной алгебры.

2 Задание

1. Используя JupyterLab, повторите примеры.
2. Выполните задания для самостоятельной работы.

3 Теоретическое введение

Julia – высокоуровневый свободный язык программирования с динамической типизацией, созданный для математических вычислений [1]. Эффективен также и для написания программ общего назначения. Синтаксис языка схож с синтаксисом других математических языков, однако имеет некоторые существенные отличия.

Для выполнения заданий была использована официальная документация Julia [2].

4 Выполнение лабораторной работы

Выполним примеры из раздела про поэлементные операции над многомерными массивами (рис. 4.1-4.2).

```
[62]: # Массив 4x3 со случайными целыми числами (от 1 до 20):
      a = rand(1:20,(2,3))

[62]: 2×3 Matrix{Int64}:
       1   1  18
       9  19   7

[63]: # Поэлементная сумма:
      sum(a)

[63]: 55

[64]: # Поэлементная сумма по столбцам:
      sum(a,dims=1)

[64]: 1×3 Matrix{Int64}:
       10  20  25

[65]: # Поэлементная сумма по строкам:
      sum(a,dims=2)

[65]: 2×1 Matrix{Int64}:
       20
       35

[66]: # Поэлементное произведение:
      prod(a)

[66]: 21546

[67]: # Поэлементное произведение по столбцам:
      prod(a,dims=1)

[67]: 1×3 Matrix{Int64}:
       9  19  126

[68]: # Поэлементное произведение по строкам:
      prod(a,dims=2)

[68]: 2×1 Matrix{Int64}:
          18
        1197
```

Рис. 4.1: Поэлементные операции над многомерными массивами



Рис. 4.2: Поэлементные операции над многомерными массивами

Выполним примеры из раздела про транспонирование,след,ранг,определители инверсия матрицы (рис. 4.3).

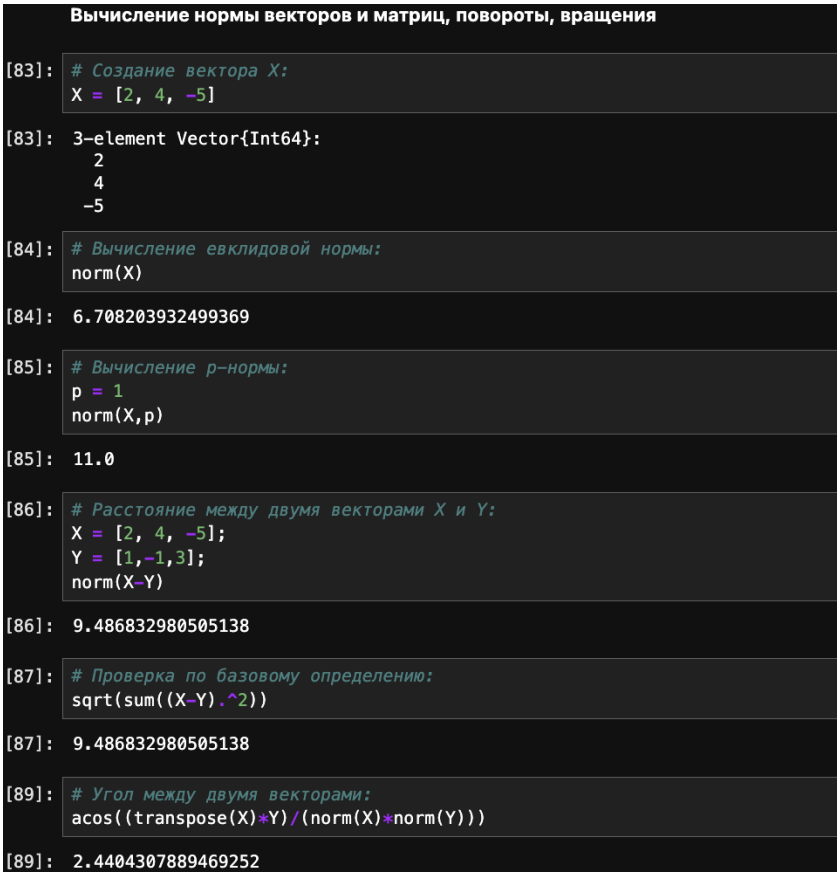


Рис. 4.3: Транспонирование,след,ранг,определительи инверсия матрицы

Выполним примеры из раздела про матричное умножение,единичная матрица,скалярное произведение (рис. 4.4).

Матричное умножение, единичная матрица, скалярное произведение

```
[97]: # Матрица 2x3 со случайными целыми значениями от 1 до 10:
      A = rand(1:10,(2,3))

[97]: 2×3 Matrix{Int64}:
       5  7  9
       1  5  5

[98]: B = rand(2:4,(3,2))

[98]: 3×2 Matrix{Int64}:
       4  2
       4  2
       3  3

[99]: # Произведение матриц A и B:
      A*B

[99]: 2×2 Matrix{Int64}:
       75  51
       39  27

[100]: # Единичная матрица 3x3:
       Matrix{Int}(I, 3, 3)

[100]: 3×3 Matrix{Int64}:
        1  0  0
        0  1  0
        0  0  1

[101]: # Скалярное произведение векторов X и Y:
       X = [2, 4, -5]
       Y = [1,-1,3]
       dot(X,Y)

[101]: -17

[102]: X'Y

[102]: -17
```

Рис. 4.4: Матричное умножение,единичная матрица,скалярное произведение

Выполним примеры из раздела про факторизацию и специальные матричные структуры (рис. 4.4-4.7).

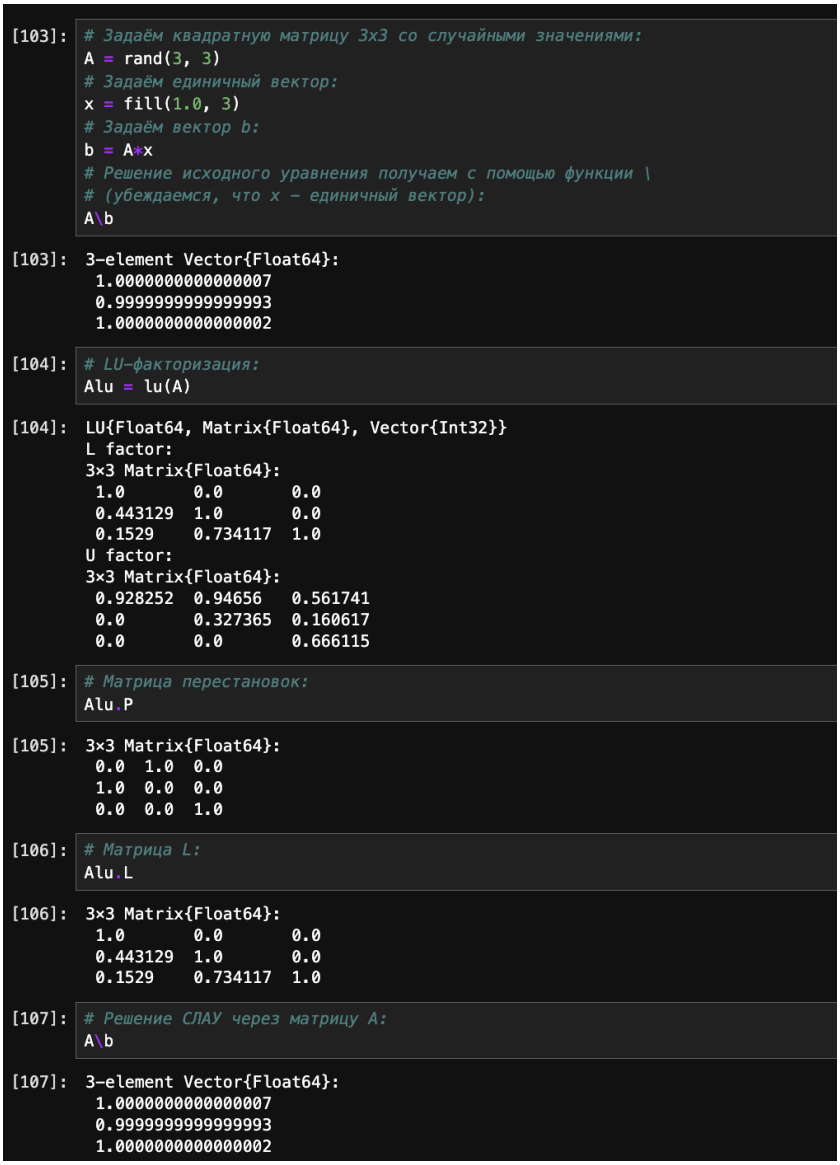
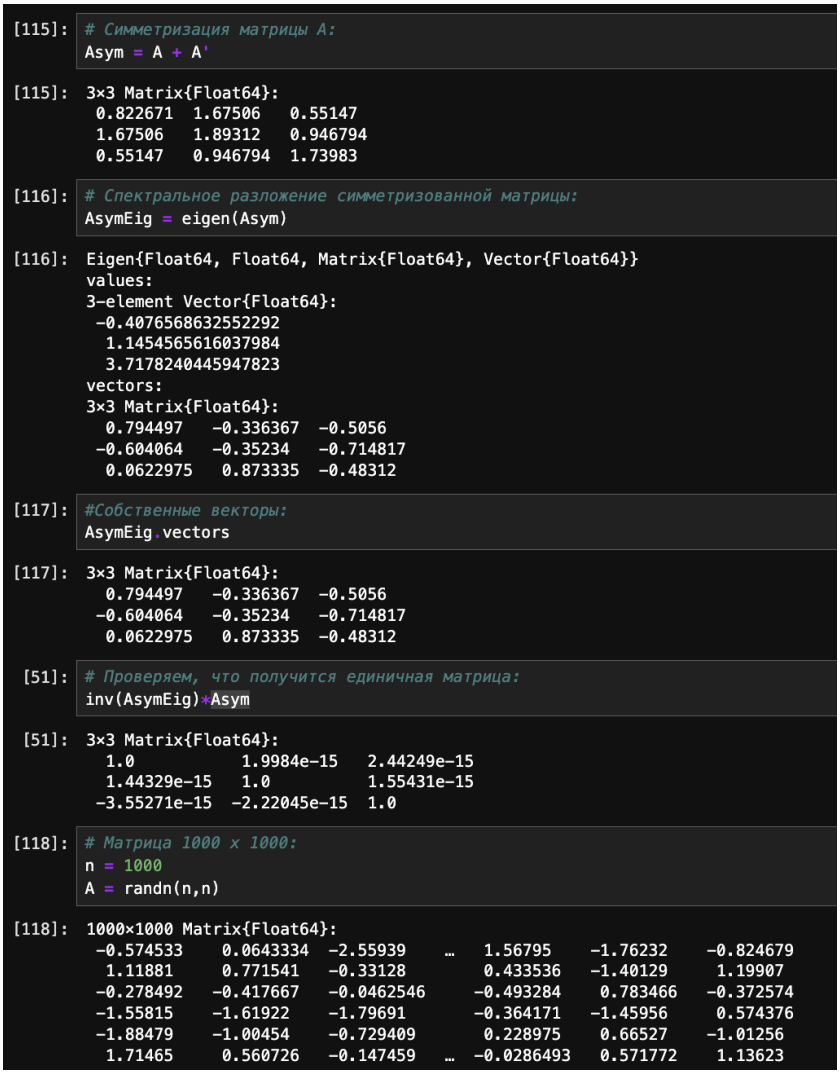


Рис. 4.5: Факторизация.Специальные матричные структуры



```
[115]: # Симметризация матрицы A:
       Asym = A + A'

[115]: 3×3 Matrix{Float64}:
        0.822671  1.67506   0.55147
        1.67506   1.89312   0.946794
        0.55147   0.946794  1.73983

[116]: # Спектральное разложение симметризованной матрицы:
       AsymEig = eigen(Asym)

[116]: Eigen{Float64, Float64, Matrix{Float64}, Vector{Float64}}
       values:
       3-element Vector{Float64}:
        -0.4076568632552292
         1.1454565616037984
         3.7178240445947823
       vectors:
       3×3 Matrix{Float64}:
         0.794497   -0.336367  -0.5056
        -0.604064   -0.35234   -0.714817
         0.0622975   0.873335  -0.48312

[117]: #Собственные векторы:
       AsymEig.vectors

[117]: 3×3 Matrix{Float64}:
         0.794497   -0.336367  -0.5056
        -0.604064   -0.35234   -0.714817
         0.0622975   0.873335  -0.48312

 [51]: # Проверяем, что получится единичная матрица:
       inv(AsymEig)*Asym

 [51]: 3×3 Matrix{Float64}:
         1.0           1.9984e-15   2.44249e-15
         1.44329e-15   1.0          1.55431e-15
        -3.55271e-15  -2.22045e-15  1.0

[118]: # Матрица 1000 x 1000:
       n = 1000
       A = randn(n,n)

[118]: 1000×1000 Matrix{Float64}:
        -0.574533    0.0643334  -2.55939    …   1.56795    -1.76232    -0.824679
         1.11881     0.771541   -0.33128        0.433536   -1.40129     1.19907
        -0.278492   -0.417667   -0.0462546     -0.493284    0.783466   -0.372574
        -1.55815    -1.61922    -1.79691       -0.364171   -1.45956     0.574376
        -1.88479    -1.00454    -0.729409       0.228975    0.66527    -1.01256
         1.71465     0.560726   -0.147459   …  -0.0286493   0.571772    1.13623
```

Рис. 4.6: Факторизация.Специальные матричные структуры

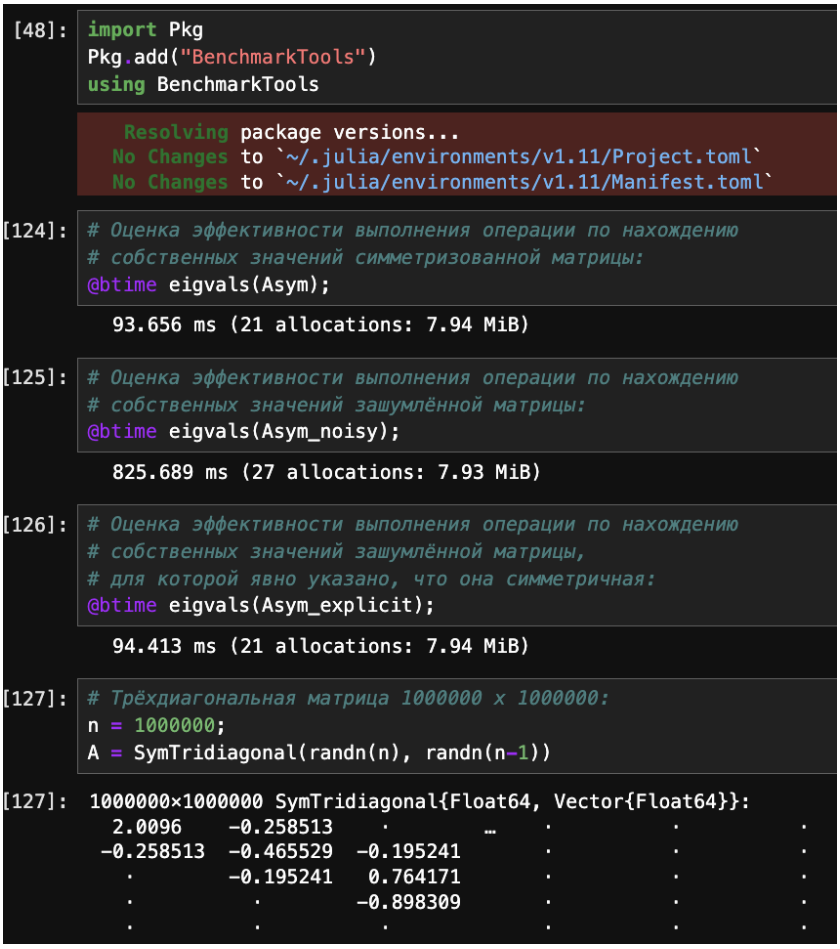


Рис. 4.7: Факторизация.Специальные матричные структуры

Выполним примеры из раздела про общую линейную алгебру (рис. 4.8).

```
[1]: # Матрица с рациональными элементами:
     Arational = Matrix{Rational{BigInt}}(rand(1:10, 3, 3))/10

[1]: 3×3 Matrix{Rational{BigInt}}:
      1//10  2//5   2//5
      4//5   1//10  9//10
      1//5   3//10   1

[2]: # Задаём единичный вектор:
     x = fill(1.0, 3)

[2]: 3-element Vector{Float64}:
      1.0
      1.0
      1.0

[3]: # Задаём вектор b:
     b = Arational*x

[3]: 3-element Vector{BigFloat}:
      0.899999999999999999999999999999999999999999999999999999999999999999999999999999999999999965
      1.799999999999999999999999999999999999999999999999999999999999999999999999999999999999999993
      1.5

[4]: # Решение исходного уравнения получаем с помощью функции \
     # (убеждаемся, что x - единичный вектор):
     Arational\b

[4]: 3-element Vector{BigFloat}:
      0.999999999999999999999999999999999999999999999999999999999999999999999999999999999999999914
      0.999999999999999999999999999999999999999999999999999999999999999999999999999999999999999914
      1.0

[8]: # LU-факторизация:
     Alu = lu(Arational)

[8]: LU{Rational{BigInt}, Matrix{Rational{BigInt}}, Vector{Int32}}
     L factor:
     3×3 Matrix{Rational{BigInt}}:
       1      0     0
      1//8    1     0
      1//4  22//31  1
     U factor:
     3×3 Matrix{Rational{BigInt}}:
      4//5   1//10    9//10
       0    31//80   23//80
       0      0     177//310
```

Рис. 4.8: Общаялинейная алгебра

4.1 Заданиядля самостоятельного выполнения

Зададим вектор v. Умножим вектор v скалярно сам на себя и сохраним результат vdot_v (рис. 4.9).

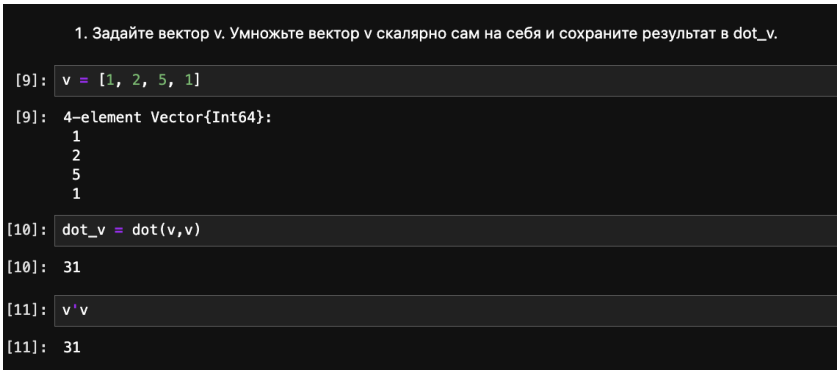


Рис. 4.9: Произведение векторов

Умножим v матрично на себя(внешнее произведение), присвоив результат переменной outer_v (рис. 4.10).

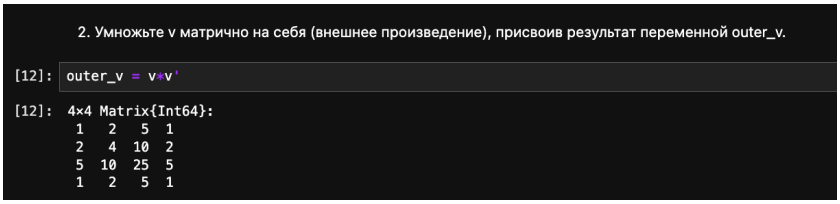


Рис. 4.10: Произведение векторов

Решим СЛАУ с двумя неизвестными (рис. 4.11-4.12).

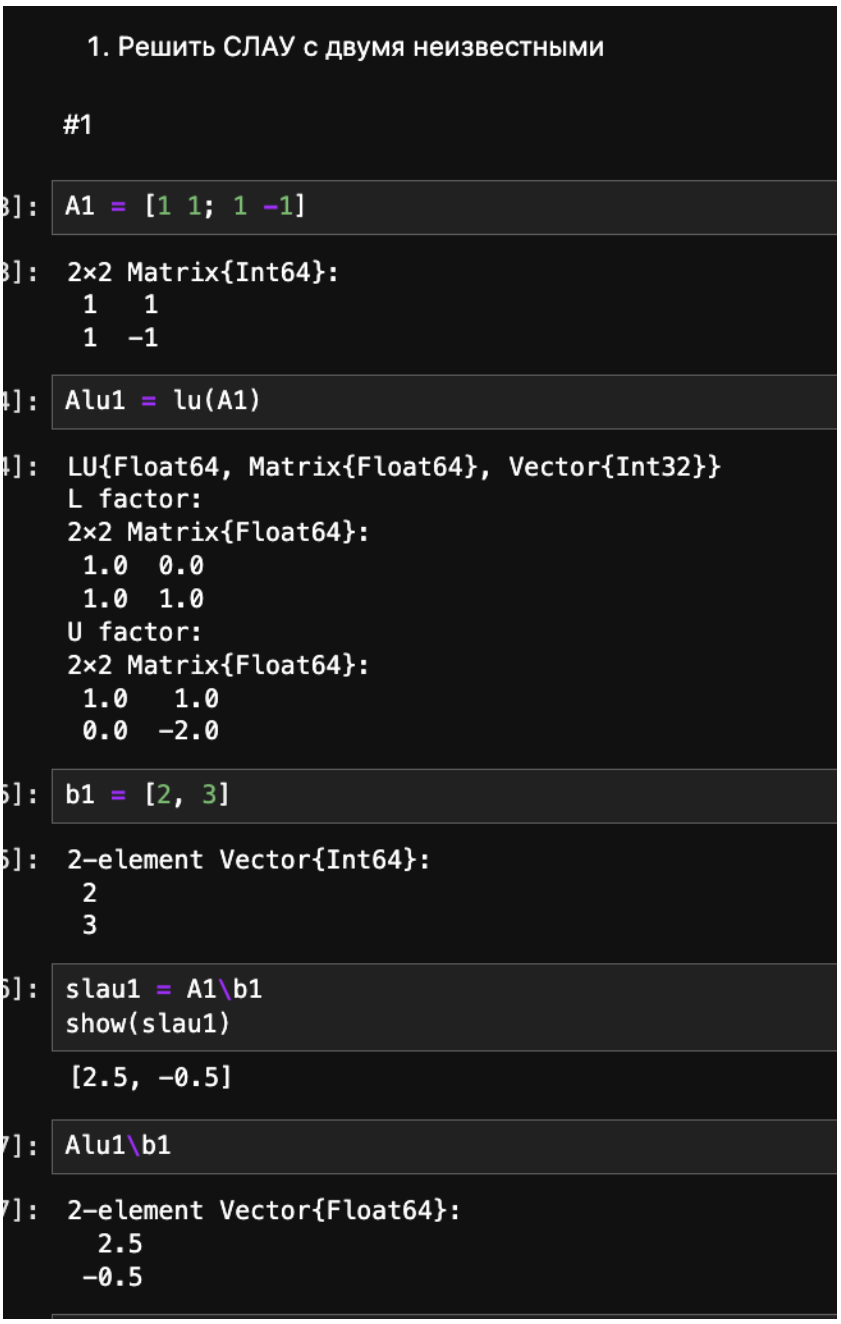


Рис. 4.11: Системы линейных уравнений

```
[18]: A2 = [1 1; 2 2]

[18]: 2×2 Matrix{Int64}:
       1  1
       2  2

[19]: b2 = [2, 4]

[19]: 2-element Vector{Int64}:
       2
       4

[20]: if (det(A2) != 0)
          slau2 = A2\b2
          print(slau2)
      else
          print("Нет решений")
      end

      Нет решений

      #3

[26]: A3 = [1 1; 2 2]
      b3 = [2, 5]
      if (det(A3) != 0)
          slau3 = A3\b3
          print(slau3)
      else
          print("Нет решений")
      end

      Нет решений

      #4

[27]: A4 = [1 1; 2 2; 3 3]
      b4 = [1, 2, 3]
      slau4 = A4\b4

[27]: 2-element Vector{Float64}:
       0.5
       0.4999999999999997
```

Рис. 4.12: Систем линейных уравнений

Решим СЛАУ с тремя неизвестными (рис. 4.13).

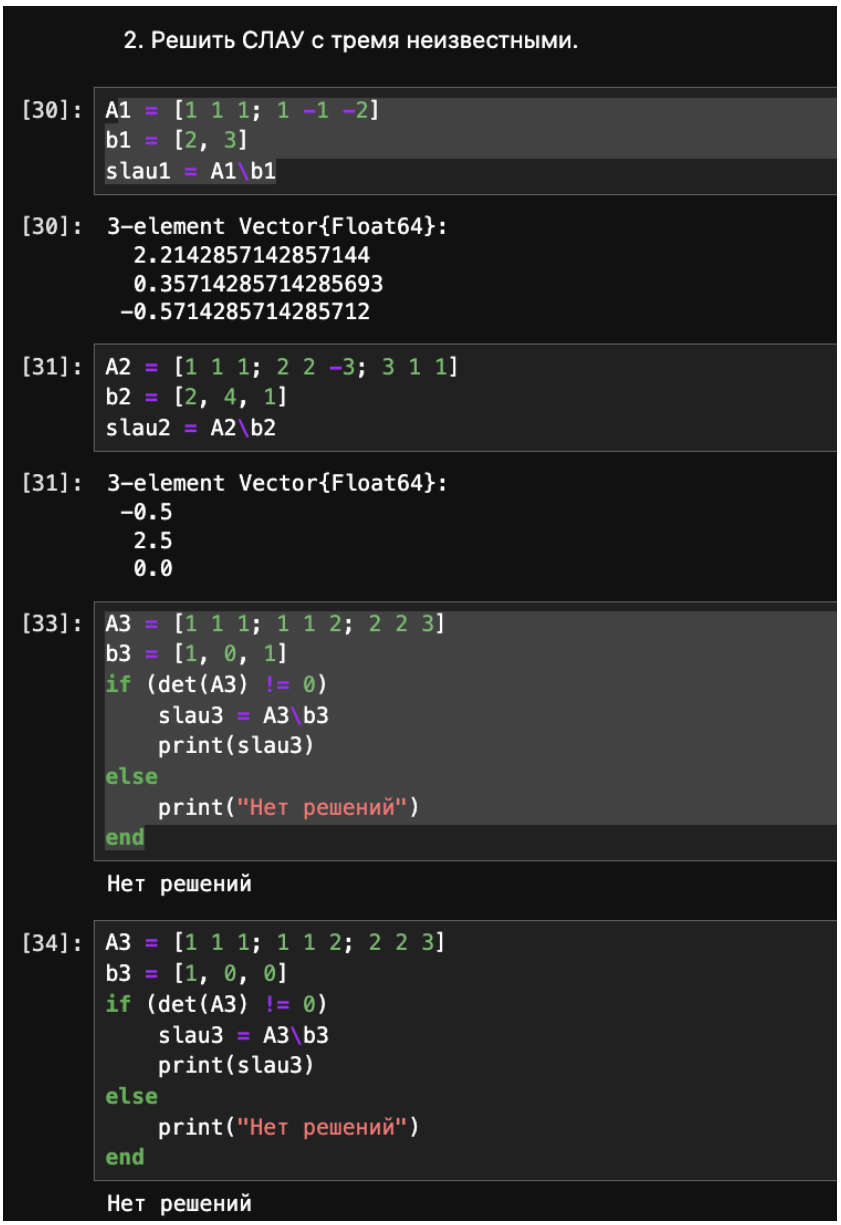


Рис. 4.13: Систем линейных уравнений

Приведем матрицы к диагональному виду (рис. 4.14).

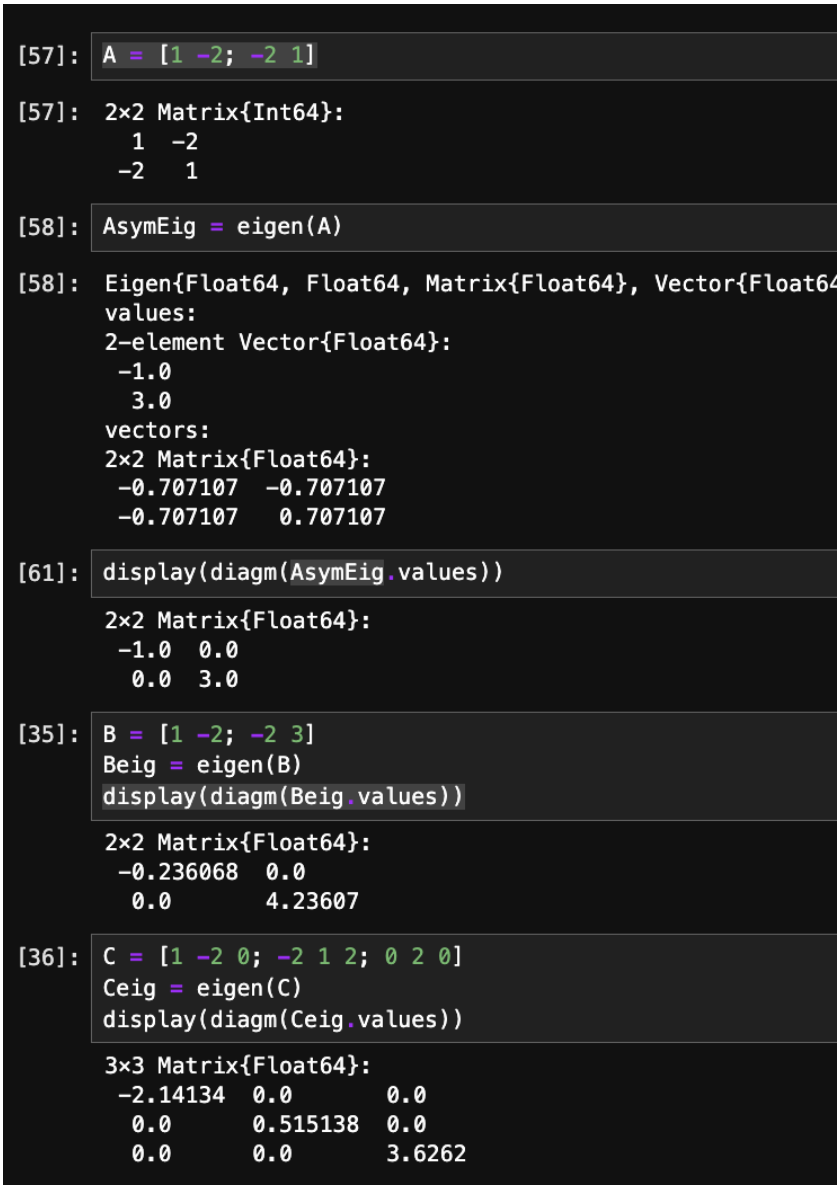


Рис. 4.14: Операции с матрицами

Вычислим (рис. 4.15).

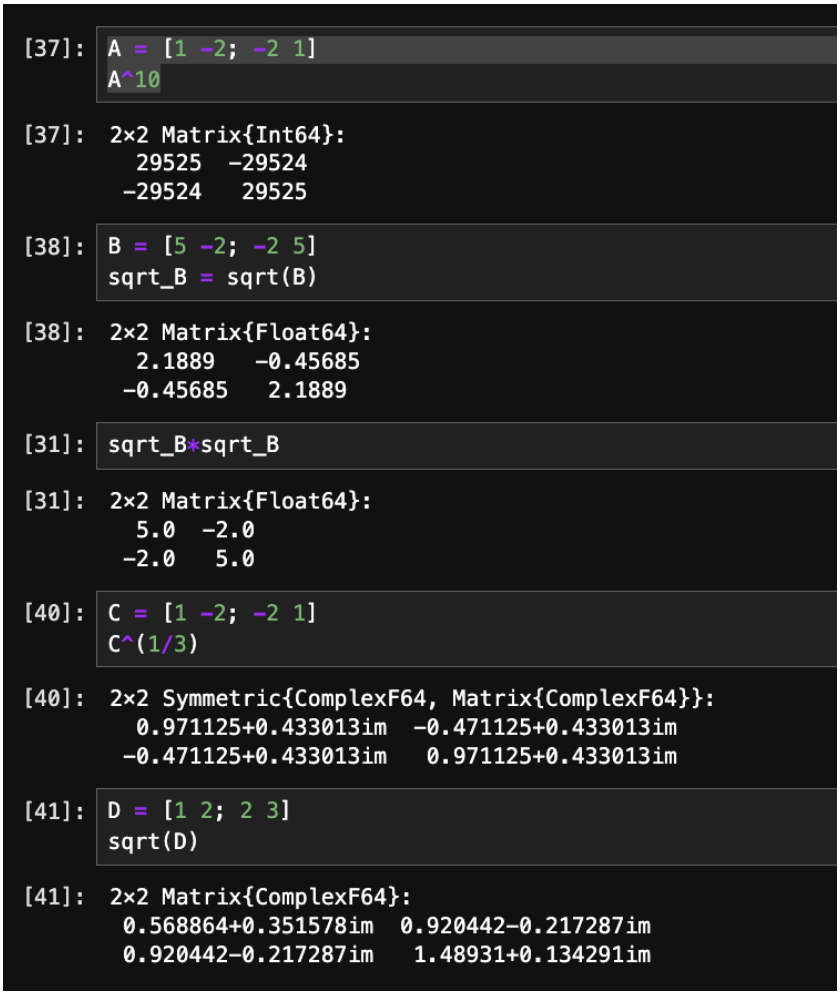


Рис. 4.15: Операции с матрицами

Найдем собственные значения матрицы A. Создадим диагональную матрицу из собственных значений матрицы A. Создадим нижнедиагональную матрицу из матрицы A. Оценим эффективность выполняемых операций (рис. 4.16).

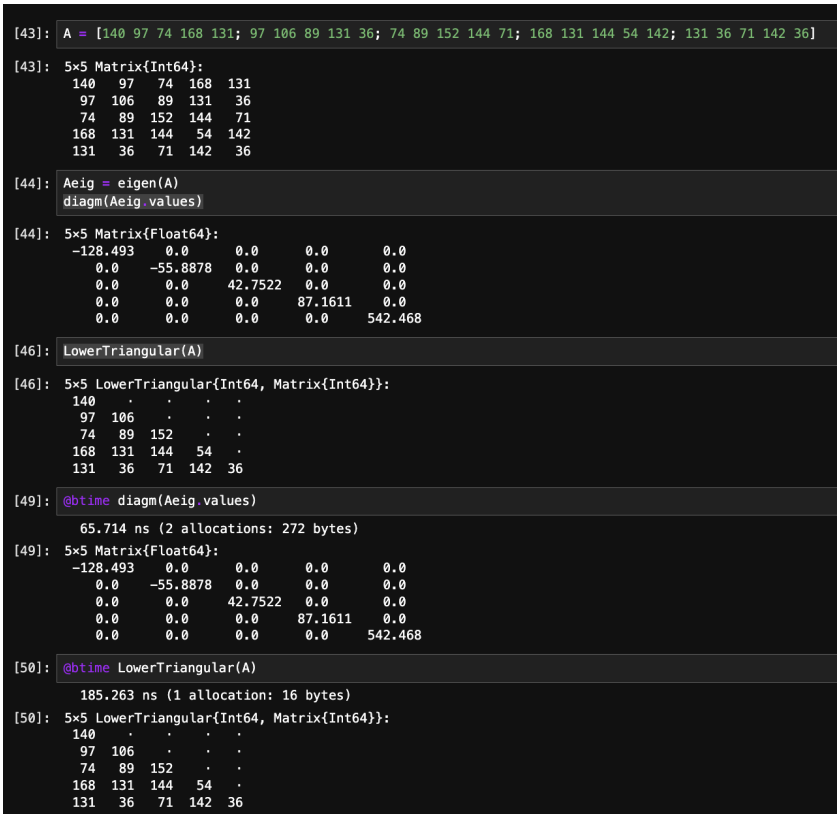


Рис. 4.16: Операции с матрицами

Линейная модель может быть записана как СЛАУ

$$x - Ax = y,$$

где элементы матрицы A и столбца y – неотрицательные числа. По своему смыслу в экономике элементы матрицы A и столбцов x, y не могут быть отрицательными числами.

Матрица A называется продуктивной,если решение x системы при любой неотрицательной правой части y имеет только неотрицательные элементы x_i.Используя это определение, проверим, являются ли матрицып родуктивными (рис. 4.17-4.18).

```
[71]: a = [1 2; 3 1]
      b = 1/2*a
      c = 1/10*a
      E = Matrix(I, 2, 2)

[71]: 2×2 Matrix{Bool}:
       1  0
       0  1

[72]: inv(E - a) # не продуктивная

[72]: 2×2 Matrix{Float64}:
       -0.0  -0.333333
       -0.5   0.0

[73]: inv(E - b) # не продуктивная

[73]: 2×2 Matrix{Float64}:
       -0.4  -0.8
       -1.2  -0.4

[74]: inv(E - c) # продуктивная

[74]: 2×2 Matrix{Float64}:
       1.2  0.266667
       0.4  1.2
```

Рис. 4.17: Линейные модели экономики

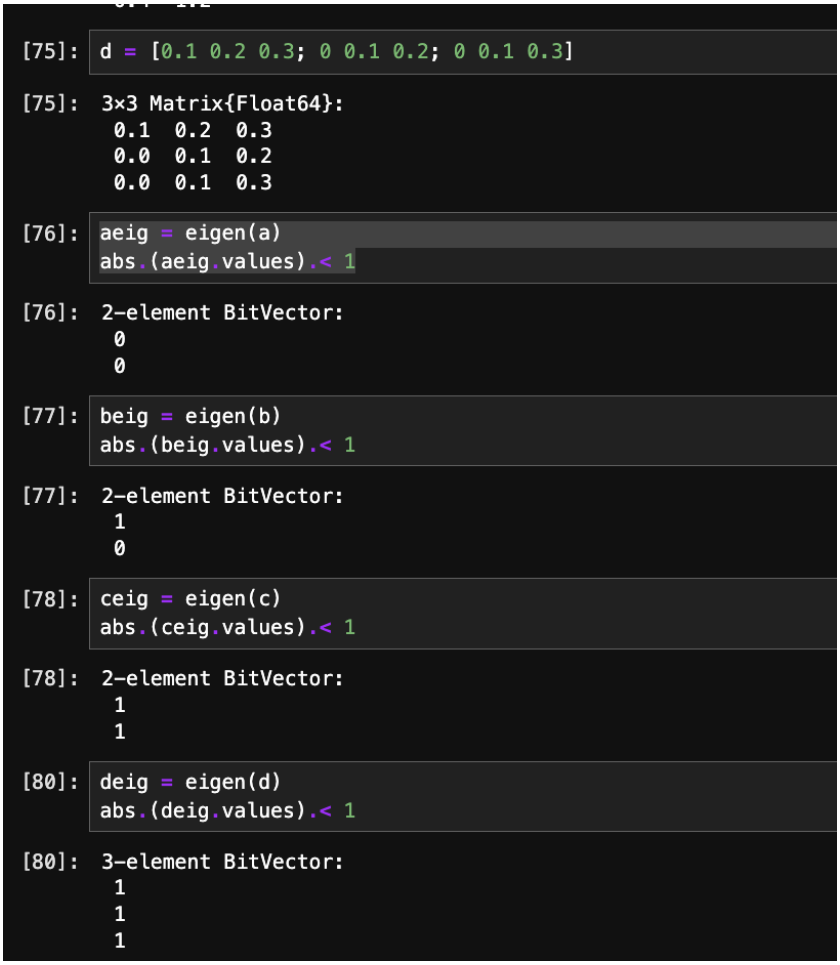


Рис. 4.18: Линейные модели экономики

5 Выводы

В процессе выполнения данной лабораторной работы я изучил возможности специализированных пакетов Julia для выполнения и оценки эффективности операций над объектами линейной алгебры.

Список литературы

1. JuliaLang [Электронный ресурс]. 2024 JuliaLang.org contributors. URL: https://julialang.org/ (дата обращения: 11.10.2024).
2. Julia 1.11 Documentation [Электронный ресурс]. 2024 JuliaLang.org contributors. URL: https://docs.julialang.org/en/v1/ (дата обращения: 11.10.2024).